

Qoder 企业级 AI Coding 培训

议程：企业 AI Coding 的痛点与对策 → Harness Engineering 等新概念科普 → SDD 规范驱动开发 → 工程最佳实践与客户实战成果 → 六件套 Skills 实操演示 → 总结与落地建议

目录

✨ 📖 第 1 章 企业 AI Coding 为什么会“翻车” ✨	-----
• 1.1 两个真实故事	-----
• 1.2 五大痛点：从个人提效到组织不确定性	-----
• 1.3 全景：企业 AI Coding 是一套七层工程系统	-----
✨ 🛠️ 第 2 章 Harness Engineering: 把 AI 关进工程... ✨	-----
• 2.1 从 Prompt 到 Harness 的三层演进	-----
• 2.2 核心公式	-----
• 2.3 User Harness in Qoder: 组件 × 形态 × 治什么病	-----
• 2.4 关键组件详解	-----
✨ 💡 第 3 章 三个新工程概念: Loop、Graph、Con... ✨	-----
• 3.1 Context Engineering: 让 Agent 读对东西	-----
• 3.2 Loop Engineering: 让 AI 跑到底还对	-----
• 3.3 Graph Engineering: 长任务是状态图，不是聊天记录	-----
✨ 📐 第 4 章 SDD: 规范驱动开发 ✨	-----
• 4.1 Vibe Coding vs SDD	-----
• 4.2 Spec 工具横向对比	-----
• 4.3 Spec 与 noSpec 的对照	-----
✨ 🏗️ 第 5 章 Qoder 企业工程最佳实践 ✨	-----
• 5.1 落地第一天：先建地基，再让 Agent 写代码（客户实战沉淀）	-----
• 5.2 十条真实的坑与规避方法（客户实战沉淀）	-----
• 5.3 提交前 Checklist 分级（P0/P1/P2）	-----
• 5.4 模型选择：任务类型—模型能力适配	-----
• 5.5 远程环境排障路径	-----
• 5.6 Skill 工程实践要点	-----
✨ 🏆 第 6 章 实战成果：阿里某万人企业客户 Qoder ... ✨	-----
• 6.1 案例背景	-----
•	-----

6.2 量化成果

- 6.3 收益从哪来

🚀🎬 第 7 章 实操演示：六件套 Skills 交付流水线 🚀

- 7.1 演示项目

- 7.2 六件套 Skills

- env-bootstrap | 项目开工前的”环境体检与建档”

- spec-writing | 把一句话需求变成能开发的规格书

- spec-review | 规格书的”质量检测员”

- deliver-system | 负责把规格书变成能跑的代码

- spec-loop-verify | 交付前的”总验收”

- ai-trace | AI 干活的”记账本”

- 7.3 交付流水线

🚀🎯 第 8 章 总结与行动建议 🚀

🚀📖 附录：参考资料 🚀



🚀🍷 第 1 章 企业 AI Coding 为什么会”翻车” 🚀

1.1 两个真实故事

故事一：修一个小bug，PR膨胀到 200 个文件。某企业客户现场，工程师让 AI 修复一个付款审核状态显示不正确的问题。功能确实修好了——但AI”顺手”格式化了多个页面文件，还调整了一个无关枚举。PR diff 膨胀到两百个文件，评审者根本无法判断哪些改动是必要的、哪些是模型自行发挥的。

故事二：单测全绿，上线必现 500。同一客户的合同管理模块，AI交付时编译全过、单测全绿。但一个未捕获的”账号不存在”异常，让普通用户提交入驻在真实环境里百分之百报500——主流程完全不可用。单测 mock 了账号服务，真实环境才会触发异常路径。

学术界给故事一这类行为起了正式名字：**Over-Editing**——模型在解决目标问题之外做了过多修改。研究表明这是当前 Coding 模型的普遍行为特征，需要外部机制约束，指望模型”自觉收敛”并不现实。

1.2 五大痛点：从个人提效到组织不确定性

个人用 AI 写代码，失败了删掉重来即可，风险集中在个人工作区。团队规模化使用后，AI的速度会放大组织协作中的不确定性：

痛点	表现	后果
需求与结果漂移	要加个筛选项，AI顺手改了枚举、排序、格式化	实现偏离最初目标，评审无法判断必要性
上下文不一致	AI 没读到”这个状态迁移必须同步更新进度”的历史约定	局部正确、全局错误
修改过程黑盒	最终 diff 包含多轮探索与隐含取舍，无事件日志	只能审代码，无法审”这段代码为什么出现”
顺手改动难约束	修 bug 扩展成顺便重构、升级依赖	review 成本与回归风险激增
验证消化能力跟不上	生成快了，测试/评审/验收没有同步变快	问题被推迟到 UAT 或生产

一句话总结：AI Coding 的企业价值不来自生成速度，来自可控交付——Spec 锚定目标，Harness约束行为，证据链支撑审查，Trace支撑管理决策。

1.3 全景：企业 AI Coding 是一套七层工程系统

AI Coding 不是孤立的模型调用问题，而是跨七层的工程系统：



七层工程系统

01 现场输入：口述需求、原型、历史代码、环境约束——天然不完整，不会自动变成可开发任务；

02 项目理解：代码地图、业务域地图、接口地图、数据地图、环境地图、规则红线；

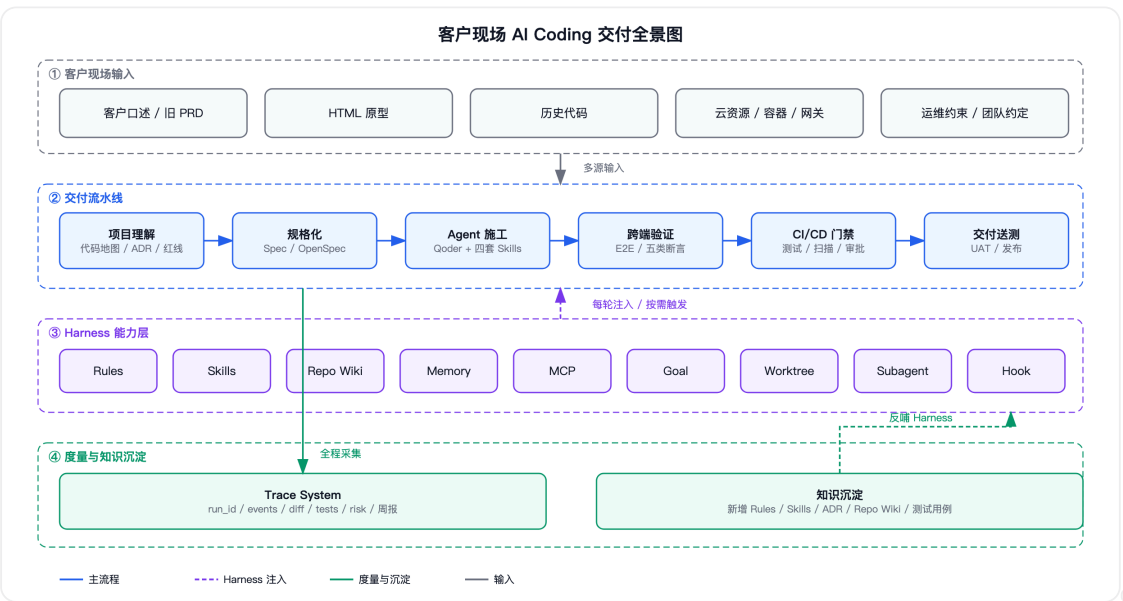
03 规格化：需求转成可审查、可测试、可归档的Spec，意图进入仓库成为工程资产；

04 Agent 执行：AI读文件、搜代码、跑命令、调浏览器、派发子任务；人的角色从”逐行写代码”迁移为”给目标、定规则、审证据、做裁决”；

05 Harness 能力：Rules、Skills、RepoWiki、Memory、MCP、Goal、Worktree、Subagent——决定 AI 是”自由发挥”还是”被工程化驾驭”；

06 验证与交付：单测、集成测试、E2E、静态扫描、CI/CD构成质量证据；交付结论不是”Agent说完成了”，而是”哪些断言通过、哪些证据可复核”；

07 度量与知识沉淀：每次 AI 开发留下trace——澄清了多少问题、改了哪些文件、失败修了几轮、哪些决策由人裁决。



✧🔧 第 2 章 Harness Engineering: 把 AI 关进工程护栏 ✧

2.1 从 Prompt 到 Harness 的三层演进

层次	解决什么	局限
Prompt Engineering	让模型听懂你说的话	每次从零提醒，不可沉淀
Context Engineering	让模型拿到充足、正确的信息	信息给了，行为仍不可控
Harness Engineering	让模型在执行过程中持续做对	——

Harness 来自一个朴素的类比：马有力量，但需要笼头、缰绳、马鞍和骑手意图才能成为可控生产力。模型有生成、搜索、推理和工具调用能力，但只有外层护栏足够清晰时，才能稳定进入企业开发流程。

Harness Engineering 定义：围绕 AI Agent 设计一套外层驾驭框架，用规则、技能、上下文、工具、隔离、目标、证据链和循环机制，让不可完全预测的模型行为进入可控的软件工程流程。

2.2 核心公式

$\text{CodingAgent} = \text{Model} + \text{BuiltinHarness} + \text{UserHarness}$

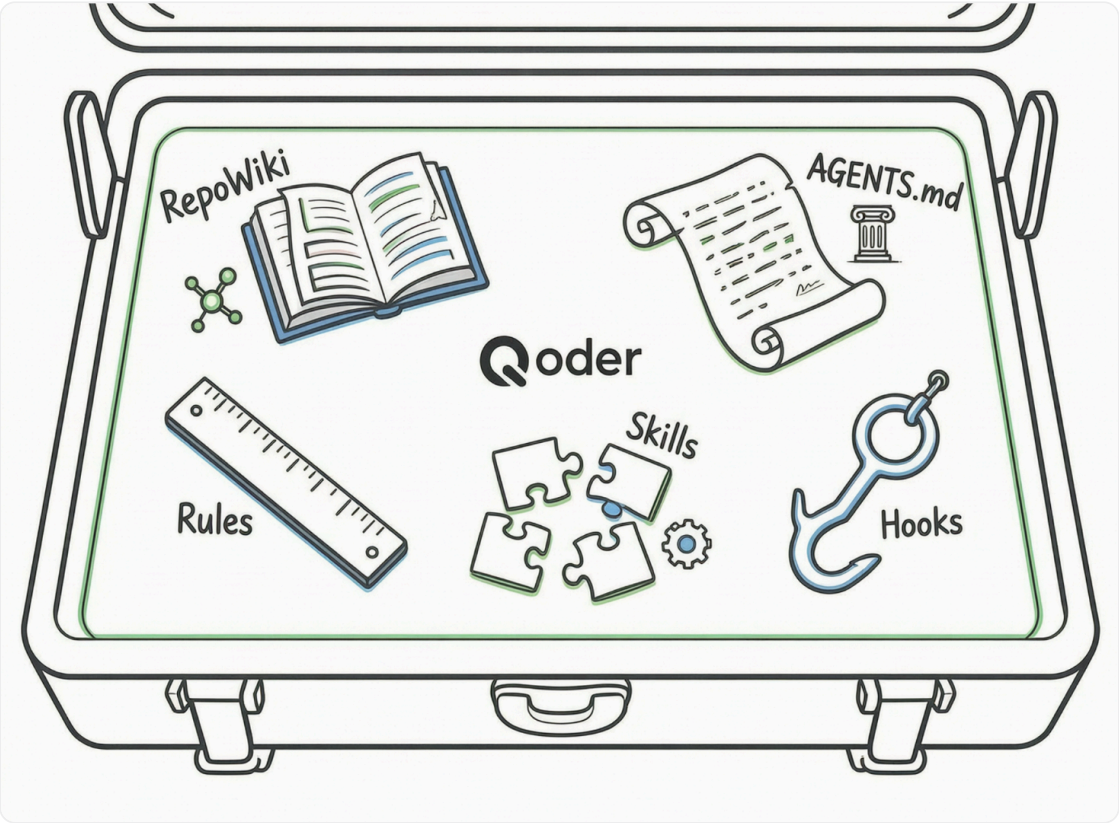
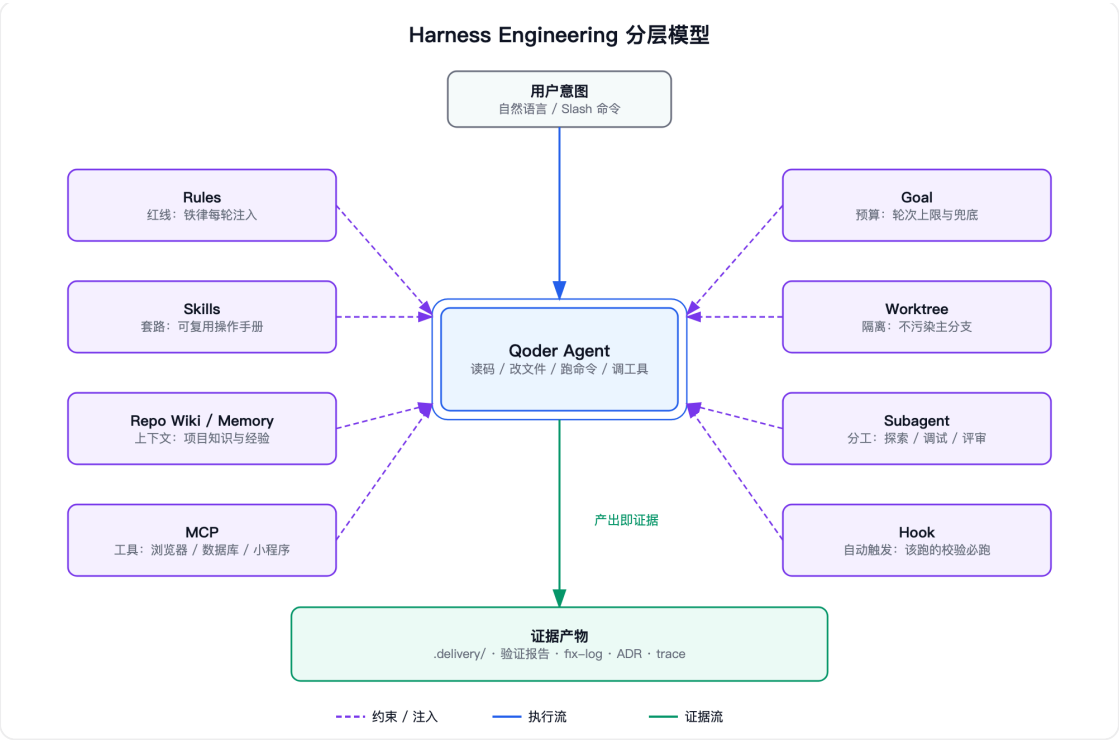
所有 Coding Agent 都内置了一套针对 Coding 的通用 Harness（系统提示词、内置工具/skills、流程编排等），同时提供扩展能力，让用户针对自身业务场景和系统架构构建**专属 Harness**。行业趋势正在从 prompt-first 走向 harness-first。

2.3 User Harness in Qoder: 组件 × 形态 × 治什么病

Harness 组件	一句话作用	Qoder 里的形态	治哪个痛点
------------	-------	------------	-------

Rules	划红线、定风格、防脑补	.qoder/rules/*.md, always-on规则每轮注入	需求漂移、顺手改动
Skills	可复用操作手册，处理某类任务不再从零思考	.qoder/skills/{name}/SKILL.md或市场安装	流程走偏、重复提示
RepoWiki	仓库结构化知识图谱，减少每轮从头探索	.qoder/repowiki/, 可git 共享	上下文不一致
AGENTS.md	写给 Agent 看的 README，项目“宪法”	项目根目录，会话开始即加载（建议 ≤200 行）	上下文不一致
Memory	跨会话保留偏好、项目约定、经验教训	Qoder 记忆库	上下文不一致
MCP	调用外部服务的统一协议，获取真实运行证据	chrome-devtools、browser-use 等	过程黑盒、缺证据
Goal	给长任务上轮次预算，避免无限跑	createGoal, 可设 turn budget	无限重试、漂移
Worktree	一个仓库多个隔离工作区，防主分支被污染	内置 / git worktree	顺手改动污染
Subagent	主 Agent 派发子任务：搜索/调试/评审/浏览器	Search / Debug / CodeReview / Browser 等	长会话上下文淹没
Hooks	生命周期节点自动触发校验，不受模型理解偏差影响	UserPromptSubmit / PostToolUse / Stop 等事件	该跑的校验没跑

核心思想是分层沉淀：红线沉淀到Rules，套路沉淀到Skills，上下文沉淀到 Memory 和RepoWiki，执行边界沉淀到 Goal 和Worktree，外部能力接入MCP。对齐之后，一次业务开发不再需要从零写长 prompt——说一句“用 spec-writing 按原型生成Spec”，Skill会自动带上项目规则、句式规范、澄清模式和输出模板。



2.4 关键组件详解

Rules (规则) —— 约束模型编码偏好，分三级落点：项目级`.qoder/rules/`仅当前项目生效；用户级`~/.qoder/rules/`对个人所有项目生效；目录级以通配符匹配特定目录/文件（如`server/**/*.java`下类必

须以 Controller 结尾），实现分层、分包、分语言的精细规范。

```
1 ---
2 trigger: glob
3 glob: *.java, *.xml, *.ts, *.tsx, *.js, *.jsx, *.less, *.css, *.sql
4 ---
5
6 MUST add a license header as a comment at the top of all source files, including new and modified files.
7
8 here is the content of the license header:
9 ```
10 Copyright 2026 the original author or authors.
11
12 Licensed under the Apache License, Version 2.0 (the "License");
13 you may not use this file except in compliance with the License.
14 You may obtain a copy of the License at
```

✦ **Skills（技能）**——场景化的上下文增强能力，自动或手动触发，把领域 workflow、SOP、团队私有知识沉淀为可复用资产。文件分用户级 `~/.qoder/skills/{skill-name}/SKILL.md` 与项目级 `qoder/skills/{skill-name}/SKILL.md`；可从 `skills.sh` 市场或 GitHub 安装开源技能（`npx skills search/add`），也可用内置技能 `/create-skill` 把 workflow 快速封装成新 Skill。



RepoWiki——类似 Karpathy 的 LLM-Wiki 思想：把仓库所有代码和文档作为原始输入，编译出仓库级知识图谱，为后续探索开发提供全局精准的上下文。本地生成后导出到 `qoder/repowiki`，可通过 `git` 与团队共享；建议关闭本地自动更新，结合 `gitflow` 和 `qodercli wiki` 在分支合并时自动更新，避免多人冲突。



Hooks (钩子) —— 由事件触发、不受模型理解偏差影响，适合配置约束治理层面的东西：文件更新后（PostToolUse）自动格式化；prompt提交后（UserPromptSubmit）自动扫描是否包含密钥等敏感信息；一轮对话完成时（Stop）调用静态代码扫描，发现问题则阻断对话结束、继续修复。

事件名称	触发时机	可阻断
UserPromptSubmit	用户提交 Prompt 后、Agent 处理前	是
PreToolUse	工具调用执行前	是
PostToolUse	工具调用成功后	否
PostToolUseFailure	工具调用失败后	否
Stop	Agent 完成响应时	否

💡 第 3 章 三个新工程概念：Loop、Graph、Context ✨

3.1 Context Engineering：让 Agent 读对东西

SWE-Explore 研究指出：仓库级 Coding Agent 的难点不只在会不会写 patch，更在能否找到足够正确的上下文。找到相关文件不等于找到关键代码区域；缺失关键上下文，比包含少量冗余上下文更伤害修复成功率。

对存量（Brownfield）项目尤其重要：Agent只读 Controller 而没读 Service 层状态机、ADR和历史约定，就会生成局部正确、全局错误的代码。RepoWiki、Memory、SearchSubagent、变更目录、历史ADR、接口文档、测试影响映射，都是 Context Engineering 的组成部分。

3.2 Loop Engineering：让 AI 跑到底还对

AI 一次做不对、多次又容易漂移——可控循环需要四个特征（四词诀）：



**有界
(Bounded)：**
预先声明轮次上限，达到上限即停止并出报告，永无无限循环；



**有据
(Evidence-based)：** 每轮判断依据是真实运行证据——数据库数值、接口返回、浏览器截图、日志，而非模型自述“我觉得可以了”；



**有档
(Documented)：** 全过程维护计划、状态、轮次报告和决策日志，可复核可回溯；



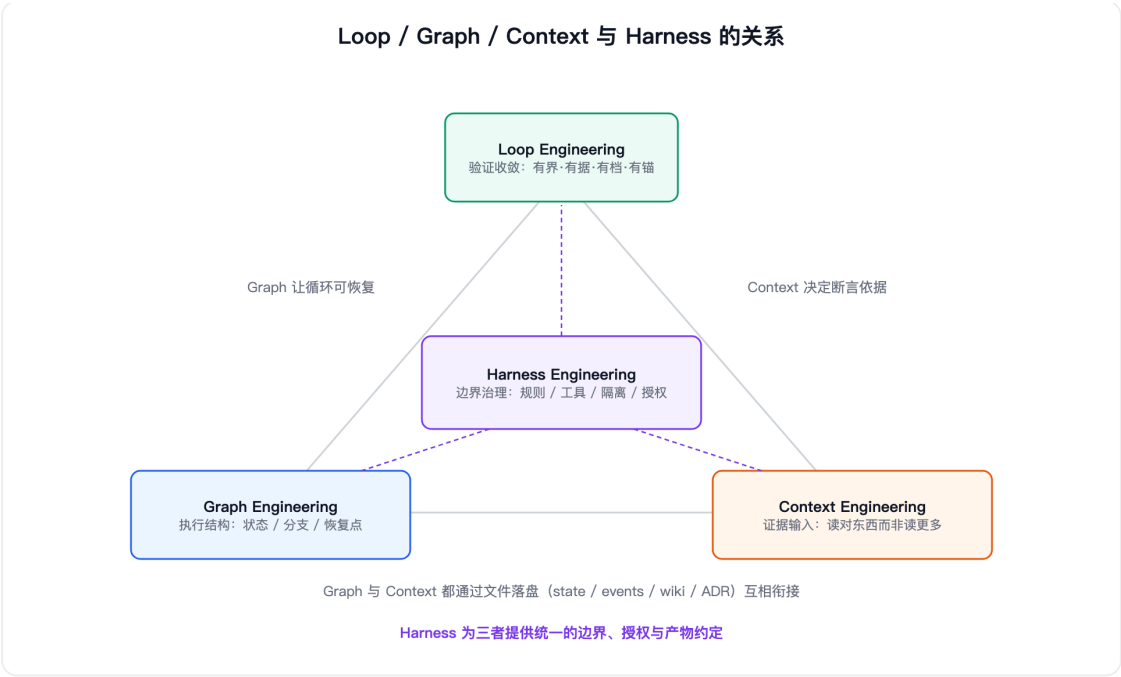
**有锚
(Anchored)：** 验收标准在循环启动前就与用户对齐，中途不允许因为难做就悄悄降低标准。

一句话对比：**Harness Engineering 让AI“跑得对”，Loop Engineering 让AI“跑到底还对”。**

3.3 Graph Engineering：长任务是状态图，不是聊天记录

Agent 做复杂任务时，在需求澄清、仓库探索、方案生成、任务拆分、代码修改、测试验证、失败修复、人工裁决之间不断切换。把这些动作显式建模成节点、状态、分支和人工介入点，团队才能知道任务当前在哪、下一步为什么这么走、失败后从哪里恢复。

落地形态是文件化的状态账本：state.json（状态机快照，支持断点续做）、events.jsonl（全量事件流）、验证报告、ADR——执行路径、状态变化、决策原因和证据产物全部可追踪。



第 4 章 SDD: 规范驱动开发

4.1 Vibe Coding vs SDD

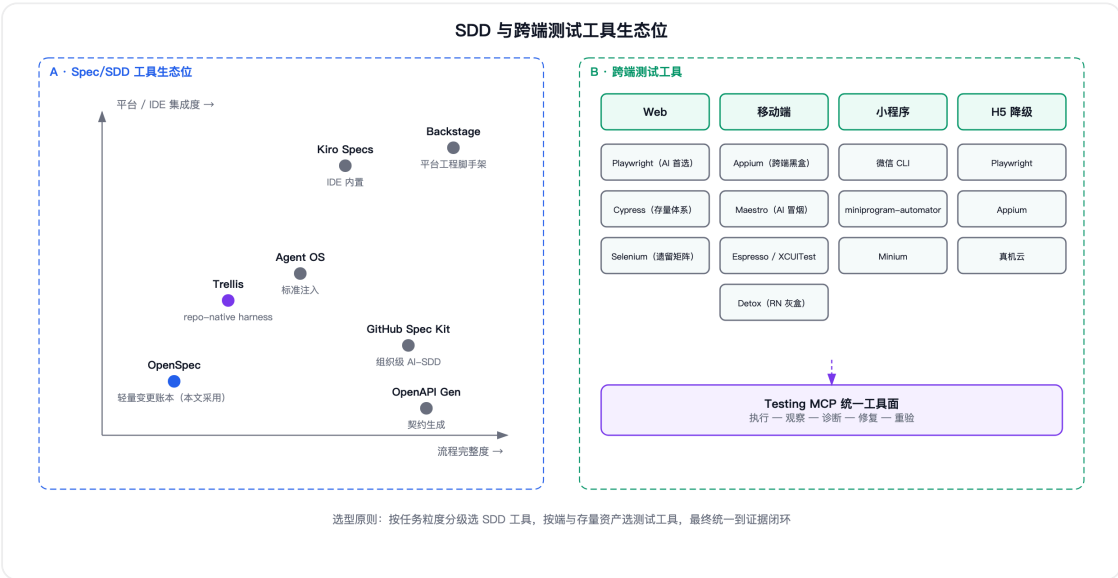
	Vibe Coding	SDD
交互方式	“我大概想要个啥，你看着办”	“我要的是这个，必须按这个来”
适用场景	快速原型、个人项目、探索性开发	企业级项目、团队协作、稳定迭代
风险	需求漂移、非功能性需求缺失、协作困难	前期投入大

Spec是一组结构化表达软件需求和功能的文档，作为人和 AI 交互的**单一真实数据源**，最终指导 Coding Agent 编码实现。典型产物：spec.md（需求与功能规格，对应PRD）、design.md（技术方案与架构设计）、tasks.md（任务拆分与执行计划）及各类详细设计文档。

以前的产出和验证边界是PRD、交互图、技术设计文档；现在的边界是进入仓库、可被 AI 和 CI 消费的 Spec。

4.2 Spec 工具横向对比

Spec 工具	Spec-kit	OpenSpec	Kiro Spec	Qoder Spec
形态	CLI（兼容多种 CodeAgent）	CLI（兼容多种 Code Agent 及 Qoder）	客户端集成	客户端集成
主动提问	否	是	是	是
使用成本	高	中	中	低
适用场景	完整制品体系，每个特性都有正式 Spec/plan/data-model/contracts	低上手成本，一条命令启动规范化流程	灵活性优先，工作流随时可迭代	低上手成本，无需提前建”宪法”



按任务粒度分级使用：小 bug 走轻量 rules + tests；普通功能走 spec + tasks；跨模块功能走 spec + plan + designreview；新服务走完整 SDD + 架构评审。

4.3 Spec 与 noSpec 的对照

对同一个一句话需求（如”给订单列表加一个异常筛选项”），无 Spec 与有 Spec 的对比：

对比维度	noSpec（VibeCoding）	有Spec（SDD）
编码过程	AI 自行推断需求边界，顺手补枚举、改排序、格式化无关文件	AI 按 spec/tasks 清单施工，diff限定在声明范围内
验收依据	无客观标准，“看起来对”	逐条对照AC（Given/When/Then）判定
生成效果	功能可能偏离真实意图，隐藏脑补需求	意图、决策、任务、验证全部落盘可追溯
评审成本	高：需逐行甄别哪些是必要改动	低：每个文件关联到具体 task





结论：好的 Spec 抵得上十次 code review。

🚀 第 5 章 Qoder 企业工程最佳实践 🚀

5.1 落地第一天：先建地基，再让 Agent 写代码（客户实战沉淀）

很多团队装上 Qoder 的第一反应是”来，帮我写个功能”。我们在客户现场的做法相反：第一天一行业务代码都不写，先把地基打好。否则没有地图的 Agent 会把一个牵一发动全身的业务能力，当成普通功能来开发。

第一步：建立八类基础资产（本次演示项目的”工程地图”就是这类资产的简化形态）

资产	目的	产物
代码地图	让 Agent 和人都知道系统有哪些模块、依赖和入口	RepoWiki 或 docs/architecture/code-map.md
业务域地图	区分核心主流程与旁路业务域	docs/domain-map.md
接口地图	梳理REST、RPC、网关前缀、各端 API	docs/api-map.md

数据地图	梳理核心表、状态字段、金额字段、索引和幂等约束	docs/data-map.md
环境地图	梳理本地、测试、云资源、容器、网关、配置中心连接方式	docs/env-map.md
规则红线	哪些模块不能动、哪些操作需授权、什么风格必须遵守	.qoder/rules/*.md
ADR 骨架	记录架构决策、偏离、降级、人工裁决	docs/adr/
变更与证据目录	OpenSpec 变更走标准 workflow，运行证据有处可存	openspec/、.delivery/

这些产物不只是“写文档给人看”，而是给 Agent 的工作坐标系：地图让它知道从哪读起、主流程和旁路怎么分；ADR让它知道过去为什么这么设计；红线让它知道哪里不能越界。

第二步：Qoder环境配置的三条实测经验



模式：默认用智能体模式（单个主 Agent 负责全程、按需自派子 Agent），适合九成场景；多个专精 Agent 并行的专家团模式实测容易上下文对不齐、结果漂移，不建议作默认。



模型：按任务类型配，不要追“最聪明”的模型。



上下文长度：默认设 200k tokens，够 Spec/design/tasks 三份文档同时在会话里；跑长交付流程、多轮验证循环时调到 1M，避免中途丢上下文导致漂移。

第三步：装市场 Skill +初始化 OpenSpec



新装 Qoder 后建议先装三个市场 Skill：



OpenSpec 初始化后，项目根目录出现 openspec/



规则双落点：.qoder/rules/*.md 承

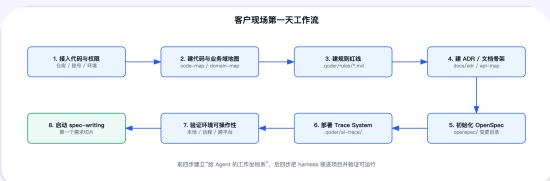
superpowers（二十多个开发方法论子 Skill 的合集）、**chrome-devtools**（管理端 UI 自动化必备）、**playwright**（端到端浏览器测试）——它们是自制 Skills 的依赖底座。

变更目录，所有变更走
explore → propose →
apply → verify → sync
→ archive 标准工作流。

载每轮注入的铁律，
openspec/config.yaml承载
变更流程约束。各项目
组规则内容不同，但落点
位置和结构可以统一。

第四步：验证环境能否被 Agent 稳定操作（最容易被忽略的一步，我们在这上面吃过亏）

✨ 客户现场常见 Windows/macOS 开发机 + 远程 Linux 容器 + 内网跳板 + 受限数据库的混合环境。如果 Agent 在 macOS 上跑通的 Skill 到 Windows 上完全失效，所谓“可复用流程”就会打折。所以环境验证不只是“能不能编译”，更是：**harness** 是否可迁移、**Skills**是否跨平台可跑、脚本能否在 PowerShell/cmd/bash/zsh 下稳定执行。



5.2 十条真实的坑与规避方法（客户实战沉淀）

#	坑	规避方法
1	长会话降智：会话跑到一百多轮后，AI明显忘了早期规则	复杂任务开新会话；上下文设 1M；Bounded Goal 声明轮次预算；关键决策及时落 ADR 让 AI 之后能读回来
2	API 契约模块改完忘了 install：编译下游报“找不到符号”	tasks 模板里改契约模块的任务后强制紧跟 install 任务
3	菜单 SQL 指向尚不存在的页面：点菜单白屏	菜单先建不挂角色，页面落地后再授权；design的 Risks 段显式声明

4	编号生成器并发冲突：并发下单号重复	Redis INCR + 数据库唯一索引双兜底，冲突重试限三次，单测覆盖并发场景
5	事务内发 MQ/RPC：事务回滚后消息已发出，数据不一致	规则明令禁止；Service 返回事件对象，事务外发送
6	Skill 触发过于频繁：随口一句话就激活 Skill 刷屏	description 写清触发条件；加”是否需要我介入”确认
7	AI 顺便优化：修个小bug，diff 变成两百个文件	规则 P0 明确不优化既有核心代码；apply前对齐文件清单；diff门禁
8	需求脑补：Spec没让做的功能 AI 顺便实现了	规则 P0 禁止脑补；proposal 显式列 Out of Scope
9	潜在同类问题未一并修：修完 A 页面，B页面下次又报	修复时主动扫全库列potential-issues，用户拍板（只列不改）
10	Skill 备份丢失：换电脑后 Skill 消失	每次迭代导出 zip 入 Git 或内网知识库

5.3 提交前 Checklist 分级（P0/P1/P2）

P0 红线（任意一项未通过禁止提交，机器可拦截的硬门禁）：符合项目铁律；未引入资金安全风险；未破坏模块依赖边界；SQL和配置为增量、不删不改已发布内容；金额字段用高精度类型；消息消费者幂等可重入；未硬编码接口地址与环境参数；未暴露密钥；SQL脚本幂等。

P1 强约束（未通过需在 PR 说明原因）：改旧代码只追加不改删、新字段带默认值；Diff仅需求范围，无顺便优化/格式化/依赖升级；接口保持向后兼容并遵循统一返回体；枚举不硬编码；提交按钮防重复；新配置进配置中心；架构和接口变更同步更新文档。

P2 建议项：符合编码规范；复杂逻辑注释说明why；核心业务单测覆盖 AC 场景；无残留调试语句；本地构建通过。

分级的意义：P0是机器可拦截的硬门禁，P1是需要人判断例外的强约束，P2是长期质量习惯。三级混在一起就都会被当成”建议”忽略，分开才有执行力。

5.4 模型选择：任务类型—模型能力适配

不要纠结”哪个模型最聪明”（模型迭代很快，排名很快过时），要建立**任务类型—模型能力**的适配意识：Spec撰写与 Review 优先中文长文档理解、结构化写作能力强的模型；长交付流程优先代码生成与工具调用稳定的模型；远程环境/容器/网关排障优先系统推理与错误恢复能力强的模型。把选型记进trace，让数据告诉你哪个组合最划算。

5.5 远程环境排障路径

同一个接口502，可能是网关路由、服务未注册、配置中心缺参、容器网络、鉴权过期、数据库连接，也可能真是代码bug。把排障顺序固化进Rules，阻止Agent”上来就改代码”的默认行为：

本地命令与网络连通性 → 容器状态 → 网关路由 → 服务注册
→ 配置中心 → 数据库连接 → 应用日志 → 最后才是代码修复

5.6 Skill 工程实践要点



实践要点

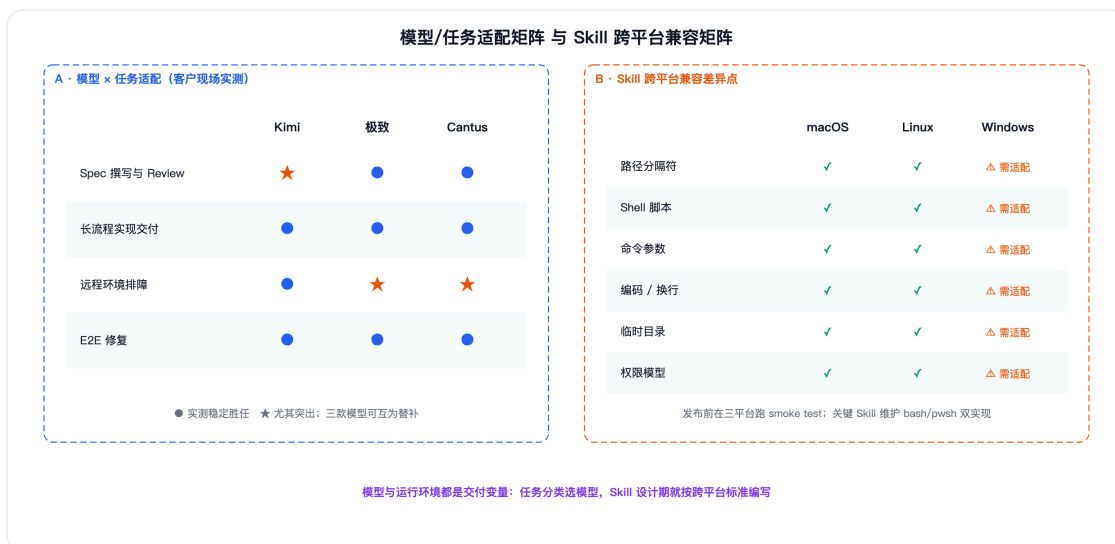
 **什么时候造 Skill**：同类任务手工做过 3 次以上；任务与项目规则深度耦合；任务顺序不能变；产出物有强结构约束。反之，一次性任务、市场已有优秀Skill、个人偏好，不造。

 **取舍顺序**：能用市场的用市场 → 差一点就 Fork 改 → 市场没有才彻底自制。

 **打磨路径**：跑一次 → 记录问题 → 改 → 再跑。成熟版特征：必需参数少、澄清分批（每批不超过 4 问）、边界写死（“我不做什么”）、输出模板化、进度可视化、失败可复现。

 **版本治理**：本地 Skill 目录默认不入Git，每次迭代导出 zip 带版本号备份；维护 Skill 清单（定位/维护人/打磨时间）；不再使用的标记Deprecated；功能重叠的合并或声明分工。

🚀 跨平台兼容：路径分隔符、Shell差异、编码换行、临时目录、权限模型、文件大小写——Skill设计阶段就按跨平台标准编写，发布前在 macOS/Windows/Linux 三平台跑 smoke test。



🌟🏆 第 6 章 实战成果：阿里某万人企业客户 Qoder 实战经验 🌟

6.1 案例背景

某万人规模企业客户，业务为面向线下网络运营方的管理 SaaS 平台。需求是把”商户入驻 → 选购 → 分次付款 → 逐笔审核 → 合同生成 → 到期提醒”全流程从线下搬到线上，覆盖用户端小程序、商户端小程序、管理端 PC 三端。

典型的 **Brownfield**（存量）场景：系统已运行多年，新模块要增量”贴”进既有多模块架构，且带着铁律进场——不触碰主流程、金额高精度处理、不破坏模块依赖边界、全部旁路新增、消息在事务提交后发送。需求 Spec 约 1200 行，拆成 5 个变更切片并行推进。

6.2 量化成果

维度	常规方式（团队估算）	Qoder +Skills（实测）	变化
Spec 撰写耗时	3~5 人日	0.5 人日加澄清	约降八成
Spec 缺陷（进入开发时）	平均 20+处	0 处（经两轮质检）	清零
C1 切片编码耗时	5~8 人日	0.5 人日	约降九成
C2 切片编码耗时	15~20 人日	1 人日	约降九成
单测覆盖率	常低于 50%	89%（行覆盖）	显著提升
Code Review 时长	2~4 小时	0.5 小时（聚焦主链路）	约降七成
进入 UAT 前遗留 bug	常 15+	0（E2E发现 5 处并全部闭环修复）	清零
P0 铁律违反	偶发	0	清零
Diff 越界（改无关文件）	常见	0	清零

口径声明：单项目实测数据，基线为团队对常规方式的估算，样本量为一，供参考而非统计结论。

6.3 收益从哪来

收益不来自”AI写得快”，来自四件事：**Spec 先行**把需求 bug 前置修掉；**规则外化到 Harness**，AI不再犯低级错误；**有界循环**让”实现—验证—修复”变成机械动作；**产物证据化**让 review 有据可查。前期成本：Skill打磨约 3 人日、规则梳理约 2 人日、流程引入约 0.5 人日——对十人团队，一次性投入支撑长期复用。

这些数字不是估算出来的，是流水线一笔一笔跑出来的。接下来的实操环节，我们用同一套方法现场完整演示一个项目的交付过程。



🌟 🎬 第 7 章 实操演示：六件套 Skills 交付流水线 🌟

7.1 演示项目

App 积分签到系统：用户每天在 H5 页面（嵌入 AppWebView）签到得积分，连续签到有加成，积分可兑换优惠券；管理端可查看签到数据与积分流水。技术栈：PHP 后端 + MySQL + H5 页面 + 管理端页面（演示环境用 SQLite 平替数据库，免安装即可跑通全流程）。

选它的原因：贴合”移动端 + PHP 后端”的典型技术栈，业务零理解成本，但麻雀虽小五脏俱全——有状态机（签到/补签/兑换）、有类资金安全红线（积分不能超发、必须幂等）、有并发场景（防重复签到）、有多端联动（H5 + API + 管理端）。

演示起点只有一句话需求：

“给我们 App 做一个积分签到功能：用户每天在 H5 页面签到得积分，连续签到有加成，积分可以兑换优惠券；管理端能看签到数据和积分流水；要防作弊、防超发。”

7.2 六件套 Skills



harness 开发 skills 六件套-clean.tgz

313.9KB

env-bootstrap | 项目开工前的”环境体检与建档”

好比新人入职第一天要先领工卡、看组织架构图：它先扫描项目里有没有规则文档、文档目录、决策记录、变更管理工具，缺什么就列清单让你勾选补齐，再生成五张”项目地图”（代码模块图、业务域图、接口图、数据表图、环境图），还会定期检查地图是否和代码不一致；内置”文档与地图随开发实时保活”的 always-on 规则，团队开箱即守同一套文档纪律。生成的文件都进 git，团队 clone 即得。



spec-writing | 把一句话需求变成能开发的规格书

给它原型图和口述需求后，它会先列出所有模糊点分批问你（每批不超过 4 个问题，带选项），比如“部分付款时订单算什么状态？”，全部问清才动笔；写出来的每条需求都带可判定的验收条件，拿不准的内容只会登记在“待定项”里，绝不替你脑补。



spec-review | 规格书的“质量检测员”

在开发开始前，它把规格书全文从六个维度逐项检查：前后矛盾、字段不一致、验收标准不可测、权限遗漏等，按P0（不做必返工）/P1（不定必扯皮）/P2（优化）分级，每条问题都写清原因和改法；你确认后才动手修改，修不好超过 3 轮就停下来请你拍板，不会无限返工。



deliver-system | 负责把规格书变成能跑的代码

有两条路：EXPRESS快车先用模拟数据快速搭出能点能看的版本让你确认体验，确认后再升级补齐真实实现；FULL完整轨走”提案-设计-任务”拆解逐步施工。全程把决定、修改记录、测试结果写进一

个账本目录，修 bug 最多重试 3 轮就停上报，不会瞎改；涉及页面的一定用真实浏览器逐个点开验证后才算完成。



spec-loop-verify | 交付前的”总验收”

拿着规格书和原型逐条对照真实环境验收：查数据库数值对不对、接口返回对不对、页面点开能不能操作到底；发现问题在授权范围内直接修，修完重新验。每个结论必须配当轮证据（截图 / 日志 / 查询结果），一个问题修 3 次还不好就停止并出报告，禁止”看起来没问题”式的口头通过。



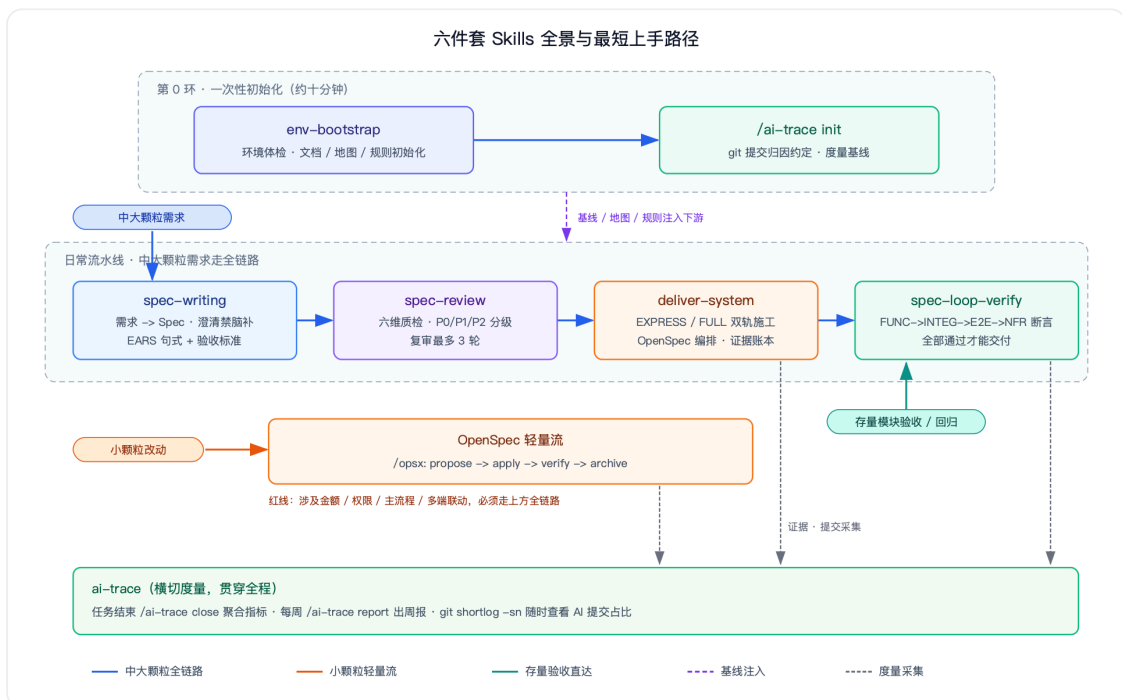
ai-trace | AI 干活的”记账本”

每次 AI 产出、经你认可的代码提交都会打上标记（AI 提交记录为 qoder 提交，责任人仍是开发者）；任务结束自动汇总：这个需求花了多久、AI 写了多少代码、发现多少问题、修了几轮；每周生成一页人效周报。



安装方式：解压到目标项目的.qoder/skills/目录（内层目录名即技能名），新开会话生效；或经 Qoder Settings 的 Skill Import 导入。依赖：deliver-system的确定性 helper 需 Python ≥3.10；diff门禁与 spec-writing 的 lint 脚本需 Node ≥18。升级请整套同日期版本替换，不要混装。

7.3 交付流水线



中间任意环节出问题都可以回到上一环节改，不需要推倒重来——AI coding 的失败不是”任务失败”，而是流水线中某个质量门没有通过。

日常使用的三条路径：中大颗粒需求走全链路（spec-writing → spec-review → deliver-system → spec-loop-verify）；小颗粒改动走 OpenSpec 轻量流（/opsx:propose（描述需求）→/opsx:apply

(执行需求) → /opsx:verify (检验成果)) ; 存量模块验收/回归直接单用spec-loop-verify。轻量流有三条红线：金额、权限、主流程变更禁用；多端联动变更禁用；不同时开多个变更。

演示过程中，每个环节都会现场打开真实产物——规格书、质检报告、交付账本、验证证据与人效周报，欢迎随时打断提问、逐项查看。

演示项目截图留存：

```
🔧 env-bootstrap — 项目体检，生成五张工程地图与文档骨架。

见 docs/env/todo-manual.md: 0 项未完成。

体检结论

✅ 17 / ⚠️ 0 / ❌ 13

详见 docs/env/env-report.md。

说明：未执行 git add / git commit。如需补齐 D1 / D4 / D5 / D6 中未生成的项，可重新运行 /env-bootstrap 并勾选对应域。

📦 11 个文件已变更 +194 -29

! config.yaml +18 -29 M
① README.md +30 A
📁 .gitkeep +4 A
MI open-questions.md +10 A
```

3. [🔗 spec-review](#) —— 对 docs/spec/App积分签到系统-Spec-V1.0.md 做质检，输出 P0/P1/P2 问题清单；逐条等我裁决后，修订产出 V1.1 定稿

docs/spec/App积分签到系统-Spec-V1.0-评审意见.md

摘要

维度	结果
总体结论	暂不通过，需先解决 P0 阻断问题后方可定稿 V1.1
问题统计	P0 ×1 · P1 ×7 · P2 ×3

关键 P0 问题（必须裁决）

P0-1 补签是否恢复连续签到天数——BR-05 与 AC-BF-01 直接矛盾

- BR-05 规定：补签「不恢复连续签到天数」
- AC-BF-01 预期：补签后「连续签到天数恢复至断签前（streak 恢复为 6）」
- 该决策会反向影响 BR-04、术语表「连续签到天数」定义、streak_after 计算逻辑及 AC-BF-01 本身，必须先行拍板。

cs > reports > MI acceptance-report.md > => # 验收报告: App 积分签到系统 V1.1 (acceptance-ready)

1 # 验收报告: App 积分签到系统 V1.1 (acceptance-ready)

2

3 - 日期: 2026-08-10 | 轨道: FULL / Greenfield | Change: 'add-signin-points-system-v1-1' (已归档)

4 - 交付标准: docs/spec/App积分签到系统-Spec-V1.1.md (评审通过, P0/P1/P2 均为 0)

5 - 绑定 commit: '5dc267e' (首个交付提交; 单测/e2e/走查证据均对应此版本)

6

7 ## 一、Bounded Goal 成功标准核对

8

9 | 标准 | 结果 | 证据 |

10 | ---|---|---

11 | 全部 tasks 完成 | ☒ 21/21 | openspec/changes (已归档) tasks.md 全句 |

12 | 单测通过 | ☒ 25/25 | 'php tests/run.php' 输出 (独立临时 SQLite 库) |

13 | ≥3 条关键场景端到端断言通过 | ☒ 6 场景 15 断言全过 | 'php tests/e2e.php' (真实 HTTP, php -S 内置服务器) |

14 | 无 P0/P1 红线违反 | ☒ 自检通过 | 见下表逐项核对 |

15

16 ## 二、AC 追踪 (22 条全覆盖)

17

18 | AC | 验证层 | 结果 |

19 | ---|---|---

20 | AC-SIGN-01..07 | 单测 (时钟注入模拟日期序列; 06 含 UNIQUE 索引存在性 + 10 次重复仅一条落库) | 全 PASS |

21 | AC-BF-01..05 | 单测 (01 含 streak 恢复=6 与后续记录重算; 边界 2001/2002/2003; 05 幕等) | 全 PASS |

22 | AC-EX-01..06 | 单测 (事务一致性, 2004/2005/2006 回滚、库存 1 两人抢兑、client_request_id 幕等) | 全 PASS |

23 | AC-ADM-01..04 | 单测 (看板双口径与直查一致、组合筛选、403、券联动); ADM-04 另经 e2e E2E-6a/6b | 全 PASS |

24 | AC-SEC-01/02 | 单测 + e2e (E2E-5a/5b, E2E-4 第 11 次 429) | 全 PASS |

25 | H5/管理端页面流程 | UI 闭环走查 8 旅程 (J1-J8) | 全 PASS, console 0 报错 |

26

27 e2e 六场景: ①签到+重复幕等 ②补签边界 2001/2003 ③兑换成功+幕等+库存 ④限流 429 ⑤401/403 ⑥管理端券配置与 C 端联动。

28

29 附录: 红线问题清单

4. [🔗 deliver-system](#) —— 按 V1.1 定稿施工。Bounded Goal: Turn Budget 25 轮; 成功标准: 全部 tasks 完成、单测通过、至少 3 条关键场景端到端断言通过、无 P0/P1 红线违反; 达上限输出未完成清单, 不硬撑

动态加载顺序 → 显式 require; ④ streak 展示口径与术语表对齐 ("最近签到日"口径); ⑥ 看板测试断言构成修正。

体验入口 (服务在线, 预览已就绪)

- H5 签到页: /h5 (demo-token-user); 管理端: /admin (demo-token-admin)
- 复现命令: php scripts/seed.php 66 php -S 127.0.0.1:8765 -t public public/index.php

证据与提交: 交付 commit 5dc267e + 报告回填 8efea95 (AI 归因 author); 验收报告 acceptance-report.md 含 AC 追踪、红线自检、假设 A1-A5 (SQLite 运行时/种子认证等偏差均已登记)。

余留项 (不阻塞验收): ④ 截图工具环境超时, 走查证据降级为 a11y 全量快照; ⑥ 生产化待办: MySQL DSN 接入、真实认证替换、Nginx+FPM 部署形态。现在你可以直接点击预览体验签到、补签、兑换与管理端完整流程。

自 32 个文件已变更 +2837 -55

MI track.md +23 A

MI acceptance-report.md +9 A

> 32 变更文件

拒绝 接受

追加需求或提问

27 / 30

5. [🔗 spec-loop-verify](#) —— 对照 V1.1 做端到端验收：数据库直连断言、curl 接口断言（含并发签到、幂等重放）、H5 与管理端页面真实浏览器验证

端到端验收完成：验证通过（32/32）

对照 Spec V1.1 的端到端验收已全部完成，一轮通过、零修复。

三层断言覆盖

① 数据库直连断言（sqlite3）——每个写操作 CP 均对拍记录/流水/账户三表：balance_after 锚点连续、余额恒等于流水累计（R1）、ref_type/ref_id 关联完整。

② curl 接口断言（24 个 CP）——含你点名的两项：

- 并发签到（CP-006）：PHP_CLI_SERVER_WORKERS=4 真实多 worker 下并发 10 请求 → 恰好 1 份 signed:true + 9 份幂等返回，DB 仅 1 条记录、余额仅 +10
- 幂等重放：重复签到（CP-005 already_signed:true 零新增）、重复补签（CP-012 返回首次结果）、client_request_id 重放（CP-018 返回同一 exchange_no 零扣减）；另有并发抢兑（CP-017 库存 1：一人成功一人 2005，库存不为负）、限流第 11 次 429（CP-024）

1...mdMI App积分签到系统-Spec-V1.1.mdMI weekly-summary.md ×

al-trace > dashboards > MI weekly-summary.md > # AI Coding 周报 (2026-W33: 08-10 ~ 08-16)

1 # AI Coding 周报 (2026-W33: 08-10 ~ 08-16)

2

3 > 数据源: `qoder/ai-trace/runs/` *1 + git 历史 (author 归因) | 生成: 2026-08-10

4

5 ## 总览

6

7 - 本周 RUN **11 个** 1 完成 / 0 BLOCKED (RUN-20260810-01 App 积分签到系统 V1.1 全量交付 + 端到端验收)

8 - 需求定稿 → 验证通过周期: **105 分钟** (周日闭环; 样本 n=1, 暂不中位口径)

9 - **AI 提交占比: 提交数 100% (3/3), 代码行 100% (+30479/+30479) **; 上周: 无数据 (首个统计周)

10

11 ## 五类指标

12

13 ### 人效 (周期)

14 | 指标 | 本周 | 上周 | 趋势 |

15 | --- | --- | --- | --- |

16 | RUN 完成数 | 1 | - | 首周 |

17 | Spec-验证通过 | 105 min | - | - |

18 | 施工-验证通过 | 59 min (22:53-23:52) | - | - |

19

20 ### 质量 (缺陷前置 / 修复轮次)

21 - Spec 缺陷前置 **11 个** (P0×1 预测矛盾、P1×7、P2×3) ——编码前全部修复, 复审 0 缺陷

22 - 端到端验收 **32/32 一轮通过, 修复 0 轮**; 交付侧单测 25/25、e2e 15/15、UI 走查 8 旅程全 PASS

23 - UAT 前置遗留缺陷 0; 验收期 SPEC-ISSUE 2 处 (口径解释性固定, 非缺陷)

24

25 ### 风险 (越界 / 红线)

26 - P0 红线违反 **0** (R1-R6 逐项自检通过)

27 - 越界文件 **0**; diff 门禁拦截 0

28 - 敏感授权 1 项 (验证造数 + 备份恢复, ADR-001, 已收尾还原并告知副作用)

29

30 ### 协作 (人工裁决 / 返工)

31 - 人工裁决 **5 次** (Spec 待决策 Q1-Q3 (streak 口径 / 兑换幂等 / 劳边界) + 验收 ADR-001 基线确认 + ADR-003 预期修正确认)

32 - 返工 0 次 (验证修复、施工后、窗口后自修: <@peak 显示口径技术事项修正并复验)

6. [🔗 ai-trace](#) close → [🔗 ai-trace](#) report —— 汇总指标, 出入效周报

close 产出 (RUN-20260810-01)

从 git 历史 + delivery-docs/ + verify-runs/ + docs/spec/ 四处自动挖掘, 生成:

- trace-manifest.json —— 结构化指标
- summary.md —— 单任务一页纸
- metrics.csv 追加一行 (可导入 Excel/BI)

report 产出

- weekly-summary.md —— 2026-W33 人效周报

周报核心结论

维度	本周数据
人效	RUN 1 个完成; Spec 定稿→验证通过 105 分钟同日闭环
AI 占比	提交数 100% (3/3), 代码行 100% (+30479)
质量	Spec 缺陷前置 11 个 (P0×1/P1×7/P2×3, 编码前全部修复); 验收 32/32 一轮通过, 零修复
风险	P0 红线 0; 越界文件 0; 敏感授权 1 (验证造数, 已还原)
> 4 变更文件	已接受

追加需求或提问

✧ 🎯 第 8 章 总结与行动建议 ✧

四句话回顾：

- 好的 Spec 抵得上十次 codereview；
- 明确的规则抵得上一百次代码规范培训；
- 有界循环抵得上一千次手工回归；
- 证据化交付抵得上一万次“我保证没问题”。

落地路径建议：



第一周：装好 Qoder + 六件套，选一个真实小需求跑 spec-writing + spec-review，感受“澄清前置”的价值；



第一个月：把一个中等颗粒需求走完整条流水线，建立团队自己的 Rules 红线和检查清单；



持续：Skill 打磨、规则更新、上下文保鲜——一次投入，长期复用。

边界声明： 以下场景不适合交给AI（或必须显式授权 + 人工复核）——生产风险操作、高资金风险的无边界处理、未澄清的需求、强主观的 UI 审美判断、跨组织审批、缺乏可运行环境的验证。



✧ 📖 附录：参考资料 ✧



🚀 Qoder 官方文档：<https://docs.qoder.com/>



🔥 QoderWork 介绍：
<https://docs.qoder.com/qoderwork/introduction>



💡 OpenSpec：<https://github.com/Fission-AI/openspec>



GitHub SpecKit: <https://github.com/github/spec-kit>



Playwright: <https://playwright.dev/docs/intro>



PlaywrightMCP: <https://github.com/microsoft/playwright-mcp>